

Learned KV-Cache Eviction for Long-Context Large Language Model Serving

AAI-590 Capstone Project

Faisal Abdul Gaffoor

Master of Science in Applied Artificial Intelligence

Shiley-Marcos School of Engineering, University of San Diego

fabdulgaffoor@sandiego.edu

Preetish Dash

Master of Science in Applied Artificial Intelligence

Shiley-Marcos School of Engineering, University of San Diego

pdash@sandiego.edu

Keywords

Inference engineering, KV cache, long-context language models, attention, memory optimization, model serving, AI infrastructure.

1 Introduction

To generate text efficiently, a transformer-based large language model (LLM) maintains a key–value (KV) cache containing the key and value tensors of previously processed tokens, thereby avoiding repeated attention computations at each decoding step (Vaswani et al., 2017; Pope et al., 2023). This cache is essential for efficient generation, but its size grows approximately in proportion to context length. At sufficiently long contexts, the cache—rather than the model weights—can become the binding GPU-memory constraint, limiting both the prompt length a device can serve and the number of concurrent requests it can process. A standard response is cache eviction: discarding selected entries to remain within a fixed memory budget. In practice, eviction is commonly governed by fixed heuristics, such as retaining a sliding window of recent tokens, preserving initial attention-sink tokens, or keeping entries that have accumulated high attention scores (Xiao et al., 2023; Zhang et al., 2023). Although effective, these hand-designed heuristics do not adapt to the task or to a model’s specific attention behavior. This project therefore investigates whether a learned eviction policy—a trained scorer that estimates the importance of each cached entry and removes the least important entries under a given budget—can produce a better answer-quality–memory trade-off than fixed heuristics.

This question is important because GPU memory is a binding constraint in long-context serving. More memory-efficient eviction could enable systems to support longer contexts, increase request concurrency, or reduce the cost of delivering the same answer quality. The problem is therefore relevant to inference-platform and machine-learning-infrastructure teams that serve long-context models under fixed GPU-memory budgets and seek to expand capacity without acquiring additional hardware. The applications and users supported by these systems would benefit indirectly through longer usable contexts and lower serving costs.

Cache-eviction methods must be evaluated on long-context tasks in which the loss of relevant cached information can reduce answer quality. This project uses public benchmarks intended for research and education, accessed through the Hugging Face Datasets library (Lhoest et al., 2021). The primary benchmarks are RULER, a synthetic needle-in-a-haystack suite that isolates retrieval across controllable context lengths, and LongBench, a multitask benchmark spanning long-document question answering, summarization, and multi-hop reasoning. LongBench includes subsets such as QASPER for research-paper question answering and GovReport for long-document summarization (Hsieh et al., 2024; Bai et al., 2024; Dasigi et al., 2021; Huang et al., 2021). These datasets are appropriately scaled for the project because they provide bounded, well-defined item sets that can be tested repeatedly across context lengths without the expense of collecting large-scale training data. The importance labels used to train the eviction scorer will not require external annotation. Instead, they will be derived from the model’s attention statistics and from interventions that measure whether retaining an entry contributes to the final prediction. In a deployed system, comparable labels could be generated from attention statistics collected from a live stream of long-context requests. The selected open benchmarks serve as controlled proxies for that request stream during development.

The project’s intended outputs are a trained KV-importance scorer and an accompanying serving harness that compares its quality-versus-memory performance with standard heuristics. The evaluation will use a small-to-medium open-weight, long-context model, such as Qwen2.5 or Llama 3.2, whose attention path can be instrumented on a single consumer GPU and later scaled to more capable hardware (Qwen Team, 2024; Grattafiori et al., 2024). We hypothesize that an attention-aware learned policy will identify cache entries that are more useful for downstream answers than entries retained by recency-, attention-sink-, or top-k-based heuristics. This advantage may become more pronounced at longer context lengths, where the blind spots of fixed heuristics are increasingly costly. Planned deliverables include the trained policy, a reproducible quality-versus-memory evaluation, the

maximum context length supportable under a fixed memory budget, and visualizations showing which cached entries each policy retains or discards.

1.1 Background Information

1.1.1 Long-Context KV-Cache Memory as a Serving Problem

Autoregressive transformer inference retains key and value representations in a key-value (KV) cache instead of recomputing attention over past tokens during each step. A key factor in memory management is that the cache grows with not only the number of generated and prefetched tokens (Zhang et al., 2023; Kwon et al., 2023), but also with the number of requests concurrently processed by the serving component. The memory issue in long-context large language models can be resolved at four different levels: Memory allocation can be improved in such a way that the cache can store values at lower precision or keep only a fixed structural subset of tokens; Alternatively or removal of entries expected to be not useful for future attention and generation can be an effective way to control growth.

In contrast to the cache compression in the previous bullet point, PagedAttention utilizes a systems-based approach than removal: Memory allocation can be optimized by organizing storage as inspired by virtual memory into blocks for KV entries such that fragmentation is cut down; This can also allow sharing of blocks between requests. Kwon et al. (2023) report a throughput improvement of two- to four-fold at comparable latency comparing this approach to currently used solutions. Further gains are realised for longer sequences and complex decoding. Hence, the discussed approach focuses on allocation optimization maintaining information while sacrificing on the compression aspect in comparison to other methods employed, such as allocation at lower precision.

A second non-eviction approach is Quantization. Liu et al. (2024) investigated key and value cache element distribution and produced KIVI, a tuning-free two-bit method which quantises keys per channel and values per token. KIVI kept nearly the same quality as in the original author's workloads, but reduced peak memory and enabled larger batches. Such studies reduce the bytes required for each cached item, than choosing which stay available for attention and as such, they may be able to complement token-selection methods

1.1.2 Existing KV-Cache Retention and Eviction Methods

In the fixed-structure approach, this takes the form of allowing a small set of initial tokens - so-called attention sinks - whose KV values persist or remain available to attention regardless of the rolling window applied otherwise. In the author's work with StreamingLLM, they found that initial tokens can be given a disproportionately high attention value even when they can be shown not to be part of attention central. Preserving this small initial set of tokens allowed the system to remain stable under rolling-window attention-caching. The authors found success preserving a certain small amount of these attention sinks. These attention sinks do not count towards the remaining token allowance as other token KV values generally would, so the initial set can be preserved irrespective of the sliding window of tokens maintained by the system during stream processing. They even tested pretraining dedicated attention-sink tokens separately, with the goal of improving this aspect of deployment. As the rule that dictates retention relies entirely on the token's position in the stream, instead of upon any separate learned importance function, the method produced by the authors here produces a cheap, useful baseline (Xiao et al. (2023)).

Attention-statistic methods try to retain tokens given that they have already had significant impact. Firstly, Zhang et al. (2023) introduced H2O. It combines tokens which either have been written to the cache recently or account for disproportionately more attention - so called attention heavy hitter. Eviction decision of H2O is formulated as a dynamic submodular problem and authors evaluate this policy using OPT, LLaMA and GPT-NeoX. OPT configs used improved their throughput relative comparison systems listed, but also reduced their latency at same batch size. Based on similar line of thought that attention-important tokens will also contribute to the attention at later steps Liu et al. (2023) came with the idea of Scissorhands. Its fixed budget manager retains central tokens (central at step $t-1$). They provide their results which show KV-cache memory reductions without any loss of quality for their settings, authors show possibility can be combined with low-precision quantization.

To pursue more adaptive solutions, both prompt- and layer-specific evidence can be adopted. Li et al. (2024) proposed a fine-tuning-free method for long-context KV caching. They observe an observation window near the end of a prompt to guess the prompt-features each attention head may use during the generation and store back only the important-clustered positions each head would choose again in generation. In their reported Long Sequence Evaluation SnapKV performs as well as the baseline and achieves a generation speedup and memory usage reduction in their 16K-context setting. As Cai et al. (2024) noted there is more than just uniform budget across layers for design, they introduced an approach of allocating different budget shares to layers according to the pyramidal-information-flow view. In their reported LongBench evaluation the layer-distributed cache performs as well as a full cache of a small fraction relative-retained cache.

Existing method or system	Core mechanism	What is retained or reduced	Evidence reported in the cited work
PagedAttention / vLLM	KV-memory allocation and sharing based on blocks, which were not supported by previous methods	Fragmentation and duplicate storage, without discarding represented information	Kwon et al. (2023) measured higher throughput at similar total latency across similar evaluated serving workloads.
KIVI	Asymmetric 2-bit quantization for KV values	number of bytes a key and value is representing	Liu et al. (2024), observed reduced memory peak and larger batches on inference workloads.
StreamingLLM	initial attention sinks, plus recent-token window and recent k tokens	Cache positions that lay outside any positional rule are listed after.	Xiao et al. (2023) observed bounded cache streaming remained stable for all their experiments
H2O	Most-attention heavy hitters and recency	Tokens which have low contribution to attention estimation	Zhang et al. (2023) obtained throughput and latency gains on reported OPT conditions.
Scissorhands	Maintenance of persistence-of-importance cache	Tokens which are not expected to remain relevant.	Liu et al. (2023) introduce fixed-budget caching compression retaining the evaluated quality.
SnapKV	Per-head: prompt observation of position and then choose	Locations not inside any of each head's chosen clusters of locations	However, Li et al. (2024) latest Transformer-based model showed comparable long-sequence performance results in its experiment and required lower memory while decoding quicker.
PyramidKV	Dynamic budget allocation per layer	More cache entries in lower layers and fewer in higher layers	However, these are now backed up by strong LongBench and needle-in-a-haystack results (Cai et al. (2024)), where the amount of retained cache reduces the context window.

Table 3. Existing approaches to managing KV-cache memory in long-context language-model inference. Reported results are study-specific and are not directly comparable because the papers use different model families, cache budgets, benchmarks, hardware and baselines.

1.1.3 Development of Attention-Guided and Adaptive Compression

Development of attention-guided compression has been driven by the finding that attention activity is uneven across token positions, heads, layers and decoding time. H2O (Zhang et al., 2023) was developed from the idea that a small selection of token positions can account disproportionately to cumulative attention, and; Scissorhands (Liu et al., 2023) captured a similar idea of temporal-persistence, in that past tokens' memory of KV states can dominate further activity; SnapKV (Li et al., 2024), a prompt observation window was deployed to identify importance patterns that exist across particular attention heads of the LLM itself; PyramidKV used a pattern of context concentration across an LLM's layers to allocate larger cache budgets to lower layers and smaller budgets to higher layers. Collectively then, these methods move beyond uniform compression by introducing conditional aspects to the KV-cache (Zhang et al., 2023; Liu et al., 2023; Li et al., 2024; Cai et al., 2024).

The proposed benefit of this family is not just about avoiding memory growth. Besides mitigating growth, attention-guided approaches attempt to avoid the answer-quality degradation that naive recency-only truncation might cause when attention is beneficial because useful bits of evidence appear more towards the beginning and in the middle of a too-long prompt. Works cited achieve superior performance under the respective evaluation designs: results are demonstrated on multiple open-weight model families for H2O, results for fixed-budget evaluation and quantization-compatibility for Scissorhands, results that span multiple long-sequence datasets for SnapKV, LongBench and needle-in-a-haystack for PyramidKV. Overall, these results provide grounds to investigate both how an adaptive cache matches against fixed positional and storage-oriented ones while reminding that benefits must be weighted by model, task, context length and memory affordance.

1.1.4 Similar Research and Production-Oriented Systems

Both research prototypes and production-oriented serving infrastructure follow similar projects. VLLM implements PagedAttention in an open-source serving system and demonstrates how the KV cache management can be applied with batching and request scheduling (Kwon et al., 2023). H2O and Scissorhands provide test time cache managers which do not require prior model fine-tuning and StreamingLLM demonstrates the streaming setting around the notion of attention sinks and a recent tokens window (Zhang et al., 2023; Liu et al., 2023; Xiao et al., 2023). SnapKV and PyramidKV continue the line work by employing prompt- or layer-dependent structure and reporting results in a set of long context evaluations involving long-document and retrieval-style settings (Li et al., 2024; Cai et al., 2024).

The evaluation setting for these systems include long-context benchmark research. LongBench by Bai et al. (2024) presents a multitask benchmark for long-context understanding and Hsieh et al. (2024) propose RULER, a controlled suit for testing long-context capabilities. Comparison is possible for traditional cache-management systems in terms of memory and throughput, as well as retrieval, reasoning, question answering and long document.

Hence, the literature considers KV-cache optimization a systems-and-quality constrained optimization problem instead of a single metric optimization: the memory solution affects serving behaviour and also the task where context must be remembered.

1.1.5 Learned Retention Policies and Semantic-Aware Compression

Compared with all previous methods which learn to select tokens, the following methods attempt to reduce the size of the attention tensor or keys and values themselves with semantic awareness.

In some cases, the retention policy itself is a learned, machine-learning component than a static ruling. Moschella et al. (2026) reformulated KV-cache eviction as a reinforcement learning problem and proposed KV Policy in which lightweight per-head agents are trained on pre-computed generation traces to order cached tokens by predicted future usefulness. The learned policy was evaluated on long-context, multi-turn dialogue, question answering, passage-retrieval and document-summarization scenarios at RULER, OASST2-4k, BoolQ, LongBench passage retrieval and GovReport which are examined in experiments carried out by the authors. This work stands directly for an example of method that provides adaptive relevance estimation (the policy is optimised from future-utility feedback while recency windows and accumulated attention score on predefined proxies of utility estimation). Semantical-aware compression is an complementary direction in the same application problem. The ChunkKV, proposed by Liu et al. (2025) which is a proposal that compresses chunks that are semantically coherent than individually scoring tokens and re-using selected chunks across the layers leading to less overhead. The approach is evaluated against LongBench, Needle-In-A-Haystack, GSM8K and JailbreakV showing that research on cache-compression now involves the long-document understand, retrieval stress tests, reasoning and safety-oriented evaluation. The landscape of relevant existing method consists of systems allocation, learned quantization, heuristic solutions for eviction, learned solutions for learnt retention as well ChunkKV's solution of compression based on semantically unit. Every solution deals with the trade-off of reducing memory cost and at the same time try to minimize runtime overhead and retain task-relevant information.

2 Data Summary

2.1 Dataset Scope and Variables

The dataset being used is a controlled RULER-like "needle-in-a-haystack", so that long-context retrieval, which is directly affected when relevant KV cache entries are discarded from memory, is isolated and thus measurable (Hsieh et al., 2024). It contains 200 records total generated from "planned context lengths" of 1024, 2048, 4096 and 8192 for the executed google colab notebook instance, from which, 30 were used as train records, 10 for validation and 10 to test for each. This makes the total size 120 and 40 records for training and validation and test records respectively; giving a 60/20/20 data split. At the current snapshot, there is only the controlled retrieval task, with LongBench subsets planned for adding as external-validity, as those are realistic questions-answering and summarization Long context tasks (Bai et al., 2024). The implementation also supports loading of data from our private Hugging Face dataset repository through the Datasets library with canonical data as well (Lhoest et al., 2021).

Table 1 is used to describe the variables presented using language that a reader would be familiar with reading. Ids and provenance are, although not model inputs, given for reproduction purposes and leakage prevention

Plain-language variable	Meaning	Role in the project
Record identifier	A label given specifically to an individual example.	It was to remain Audit, not used for any predicting features.
Original source identifier	The id provided by the generator or the external benchmark.	It allows for track and trace and grouped splits.
Source collection	Record from Controlled Corpus, LongBench or else?	This ensures we control for the obvious difference in subject domain and task difficulty.
Source subset	The benchmark subset or generation template associated with a record.	Allowing for stratification and sub group evaluation.
Data partition	Training, validation, testing and external testing for example sets.	This will stop there being contamination between the evaluation and also set the limits for the development.
Task type	The long-context capability is currently being tested on single-answer retrieval by any existing benchmark/record.	It lets you focus on one particular task more precisely once the project progresses
Planned context length	The number of context tokens asked for inference.	The length of the experiments is the primary condition that defines the cache requirements.
Measured context length	Number of tokens for the counting rule in question.	Used for memory accounting and detection of generator or tokenizer drifts.
Answer location	Percent position within the context of the location of hidden answer.	Tests recall for late evidence.
Number of hidden targets	Number of relevant answer bearing entities in the context.	Difficulty variable - fixed at one for the current snapshot.
Generation seed	Value to use to make generation deterministic.	Provenance only; was excluded from the feature matrix.
Long context passage	input : text given to the language model before the prompt.	The original string from which token-level examples will be extracted.
Question or instruction	The prompt asks the model to output the hidden key.	This is raw model input.
Reference answer	Correct key used for scoring response.	target that is not available to the model during inference.
Token-counting method	The rule or tokenizer you use to determine the context length.	Record the actual possibility to occur for possible measurements to shift.
Generator version	The version of the generation / normalization procedure.	It supports the ability to reproduce and also to version control a dataset.

Table 1. Variables present in the current canonical dataset and their project roles.

In the later policy-evaluation table, five fields- the eviction policy; the retained cache fraction; the measured cache memory; the response quality or correctness or error; and the serving performance variables- will be added. These are not present in the current data-summary snapshot. Until reported via these additional fields, these project components are treated as "planned variables" (design expectations) than as "observed results" (empirical findings)..

2.2 Data Quality, Cleaning, and Pipeline Controls

A notebook kept an immutable raw snapshot of data and applied rules for validation in a deterministic way, before sending "canonical clean" table. The data audit covered: required text checks, the replication of content, cross of content duplication from one partition to other, the location of a reference answer, the drift context in length and the legitimacy of answer location. The number of non-existing texts that must exist was found to be 0 or there was 0 cases of existing duplicates and none such case was found to be from one partition replicated on any other position. Similarly, there were no retrieval which did not reference an answer in context and no case the where location of answer was found outside the interval of valid [0%-100%] was seen. Zero cases were there for a 5% difference between planned and measured length of document. No records were moved from the dataset of 200 records. So no

quarantined records created. Further more, the optional type meta data would be assigned a value of un known than impute in a statistical manner because such fields specify provenance than describing continuous variables.

Audited issue	Result	Notebook handling	Developed-pipeline control
Missing context, question, or answer	0	-Quarantine unusable rows and retain the raw immutable record.	A required field schema should be implemented prior to partitioning.
Duplicate content	0	Hash normalized content, return first valid occurrence.	- stable hashes and idempotent ingestion.
Content crossing partitions	0	Quarantine overlap, rebuilds did hit splits.	Deduplicate before grouped partitioning.
Reference answer absent from context	0	Quarantine impossible to evaluate thing.	The answer has been included after final curtailments.
Length difference above 5%	0	Flag up moderate drift than automatic rejection.	Pin production tokenizer and monitor drift by version.
Answer location outside 0%–100%	0	Quarantine any invalid positions.	Ensure range constrains are within our canonical sche_x0002_ma.

Table 2. Data-quality results, implemented handling, and recommended production controls.

Most probable sources of future data problems: cross-benchmark schema differences, incomplete extraction, templating or truncation bugs, repeated download or generation retries, splitting before deduplication and approximation counting rule vs model's tokenizer disagreement. A (more) developed (than the Jupyter Notebook was) machine-learning pipeline should embrace schema validation, immutable and versioned raw storage, deterministic generation, content hashes, deduplication before partitioning, group-aware splits, pinned tokenizer revision, automated assertions and build-stopping data-quality tests.

The dataset should remain private on the Hugging Face Hub, because long contexts and reference answers are evaluation material and only model artifacts and aggregate metrics after they have been thoroughly reviewed should be published. Regularization will not be applied at this data-summarizing stage because no predictive model will be fitted in the attached notebook. When the token-importance scorer is trained, regularization will be chosen for on the validation partition, not instead of data cleaning. Candidates worthy of considering: weight decay, dropout, early stopping, capacity controlling through a shallow architecture for the lightweight scorer. Class weighing or resampling options should be examined only after token-level keep/evict label distribution has been measured because balancing could spoil the budget, before we know labels true distribution for them..

2.3 Exploratory Analysis and Relationship to the Project Goal

The experimental design is balanced across two dimensions simultaneously: length as well as data partition. The same context-length distributions of fifty records per planned length (all same 30/10/10 for training, validation and test partitions, respectively) is ensured across all four lengths. Having the respective data partitions contain the same context-length distributions, hence also the same length mean of 3068, helps in ensuring that any advantage achieved by one policy over the other is actually the result of the changed parameter and not due to shorter examples/articles in the respective partition compared to its counterpart.

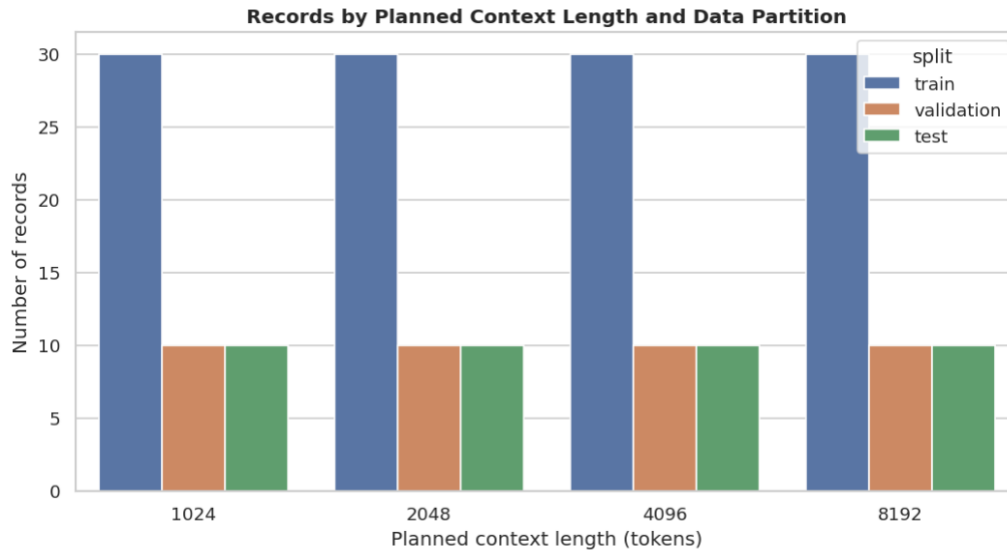


Figure 1. Records by planned context length and data partition. Each length condition contains 30 training, 10 validation, and 10 test records.

The answers range in location from ~2% to ~98% across the long-context and positioned at the early, middle and

late parts of the input, as designed (Figure 2). This variable has specific relevance concerning cache eviction where a simple recency policy may preserve the end of a prompt with the required evidence while throwing away the useful information that went before. Position coverage allows the final evaluation to test whether the learned scorer can preserve semantically meaningful tokens without simple reliance on position alone. At this junction there is found no particular correlation between position and length (Spearman $\rho \approx 0.03$) meaning the effects of position and length can be studied without material confound.

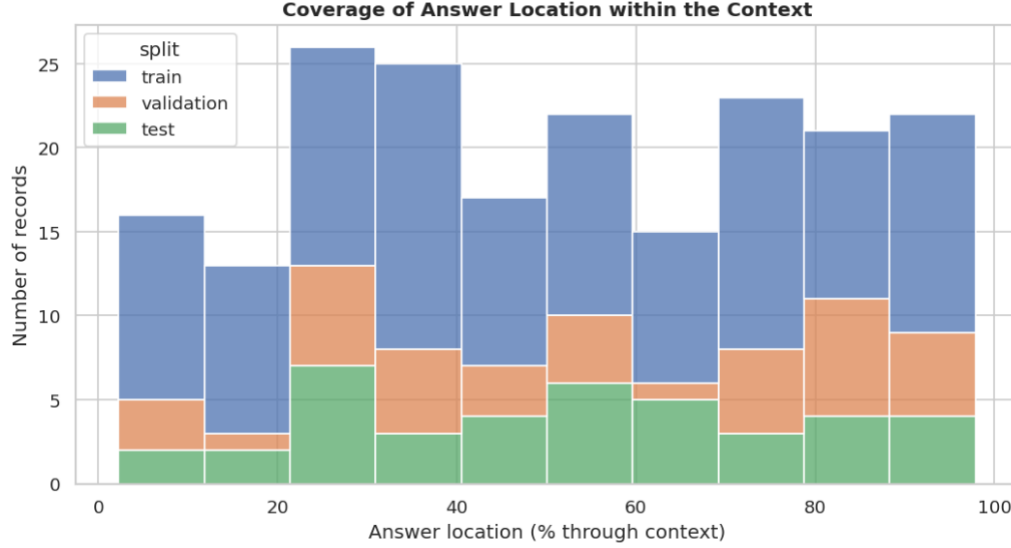


Figure 2. Coverage of the hidden answer location within the long context. The design includes early, middle, and late evidence positions.

The **context length** parameter affects how much is stored to cache and indicates how effectively the state representation is compressed by the summarization model. Meanwhile, **answer location** represents a target problem that must be solved. The remaining three values, namely, the **passage**, **question** and **reference answer** contain the inputs and evaluation targets. The **eviction-policy** and **retained-budget** are placeholder variables for new treatment types that will be specified at later steps. Finally, with the **cache bytes (measured)** and **task quality (measured)** will determine the two axes of the **quality vs. memory optimization boundary** that is the goal of our problem. The rest are administrative values such as **record (unique identifier)** and **hash, seeds, splits**, that ensure reproducibility. Such values must *never* be used as features for the models, unless models are intended to memorize their exact training dataset entries by association. For convenience, the implementation of such data parsing steps and the entire workflow are available in a Google colab notebook.

2.4 Relationships Among Variables and Modeling Implications

The **strongest** relationship found is between planned and measured context length. These have a Spearman rank correlation coefficient value of 1.00 because each time I requested I longer context, the length I obtained was proportionally longer than the last time I requested one. Because of the nature of the generation template each time I used it the length of the string provided to me was roughly 4 whitespace-delimited tokens less long than the one I had originally provided. Both measured and planned context length are **negatively** rank correlated with resulting drift percentage (ρ equals negative 1.00 - drift is proportionally smaller to context length). Given the context of a 4-token length difference this effect is mechanical than indicating a phenomenon resulting from behavior, something

3 Methods

3.1 Pipeline and the thought that drove it

The problem in this study - KV-cache retention - is treated not as a language model retraining, but as a long-context inference task under resource constraints. The solution is arranged as a staged Google Colab suite where cache accounting and state operations are proven correct before benchmark measurements are taken. First notebook is devoted to some theoretical foundations and exploratory data analysis, the second to core test of eviction mechanism and the third is used in a T4 oriented RULER baseline sweep. Any next stages, be it learned policy or some tests that are effectively locked, are considered a separate experimental phase than a continuation to exploratory development, making change of policy, benchmark setting or test outcome being change-able after having looked at it less of an issue.

A named run root with subdirectories for manifest, benchmark manifest, cache snapshot, attention profiles, traces, results, figures, logs and tests are created during run. The environment for the notebook is determined using some pinned package versions, constant random seed (590), no model or dataset version changing and GPU available will be detected runtime and one JSON file will be included for the runtime manifest. The project is implemented in Google Colab. Artifacts must be manually copied from colab working directory which is temporary, to the Google Drive directory of project, at the end of each successful runtime.

3.2 Model, runtime tiers & cache accounting

For its hardware-appropriate tiered instruction models this experiment uses open-weight Qwen2.5 models. The tier T4 uses Qwen2.5-1.5B-Instruct in 4-bit loading mode, whilst the bigger RTX class and H100 are left to be used to measure the learned-policy and locked-test phases respectively. The hardware (asked, detected card, active card, fallback, revision, python/torch version, seed) are recorded with the runtime manifest. Even in the case the asked and detected cards mismatches it is stored on the manifest and a measurement will never be wrongfully labeled as a measurement on the requested device.

Before the benchmark sweep, in foundation notebook KV-cache accounting was GQA-aware. Key and value attention head count, hidden-head dimension, number of layers, batch, retained-token count and tensor datatype which is all fetched inside model are used to determine analytic cache count and compared with actual physical cache payload through formula gate having two percent tolerance. Logical and physical cache length is handled distinctly in the library where context length keep growing whereas cache length can shrink post eviction since the policy that maintains correct logical token may incur smaller physical KV-cache budget.

3.3 Cache Correctness and Policy Baselines

The eviction mechanism is tested and proved working properly before it can be used for benchmarking by cache-core notebook. Pre-fill the model cache with chunks, construct the state from physical positions, apply each candidate policy at budget, record its retained lengths. Non-full policies respecting physical-cache budget (monotonic retained positions; no policy returns empty retention set). Fill-cache equivalence testing that apply fill policy is identical to non-evicted with no change to prefill logits, greedy next-token and decode logits. Forced compaction boundary + 3-step greedy streaming-decode test: decode continues with sense-making logical position and healthy physical cache.

Our policy baseline to beat consists of full cache, FIFO, seeded random, uniform, sink-plus-recent, attention to Top-K and H2O-style heavy-hitter-plus-recent retentions. Full cache is the uncompressed reference. Every example with each non-full policy gets the same discrete cache-budget grid so we compare quality of retentions not with unequal amounts of memory. Sink tokens are handled, as a reserve for recent tokens, to be an explicit constraint of retention than unrecorded, implicit details of implementation, so we get later learnt retention to compete with a transparency Reproducible base line of policies (Xiao et al., 2023; Zhang et al., 2023).

3.4. Benchmark Data, Sampling and Attention Profiling

Notebook 00 uses a deterministic controlled RULER-like single-needle retrieval corpus of 200 records: 50 records at each planned context length (1,024, 2,048, 4,096 and 8,192 tokens). The fixed split is 120 training, 40 validation and 40 locked-test records (30/10/10 in every length stratum). Each record contains one answer-bearing needle at an early, middle or late location (approximately 10%, 50% or 90% of the context), together with stable IDs, planned and measured token counts, a reference answer, and a generation version.

Notebook 00 audits the corpus before modelling: it checks split balance, required fields, duplicates, answer presence, answer-location bounds and token-length drift. The executed audit found no violations and measured lengths within the 5% tolerance. A short eager-attention pass is used only for diagnostic profiling; the evaluation path uses attention-free inference. LongBench remains a separately labelled future external-validity extension and is

not part of the active controlled-corpus results.

3.5 Gates, Measurements and Reporting Boundary

The T4 benchmark sweep is opt-in; only when a confirming acknowledgement has been received from the operator concerning GPU availability, free memory, disk space, and desired experiment allocation does it become enabled. A suite will write an explicit `requires_larger_gpu_or_smaller_context` record rather than fabricating an artificial result when the requested context or policy combination has not been able to fit onto the currently active hardware the suite was requested on. Similarly, when we experience CUDA out-of-memory exceptions, we convert these into corresponding status records. This is especially important when requesting against a 15-GB T4, where many repeats across the configured benchmark grid could need nearly the full allocation for the full run of 100 benchmark repeats on the specific model requested by our suite, and where a 95% model allocation means a successful, reliable run is unachievable.

For succeeded executions, are recorded: prompt-token, (logical, physical) retained-token, (analytic, physical) KV-bytes, eviction budget, tokens generated, policy identifier, benchmark row identifier and output status. The T4-visualization is restricted on entries labelled observed; hardware-limit and error ones stay visible in the CSV but vanish from any performance curve. The substring-match among the notebook obtains is retained (only as a transparency-preserving retrieval diagnose) but never meant as a substitute for performance-faithful scores for benchmark. The project refrains from introducing an unentered run (or a max-hardware row or a diagnostic proxy instead of scoring for benchmark (quality) as (if it is) a finished benchmark (result)).

The learned scorer is evaluated only after the cache-accounting and heuristic-baseline stages have passed. Training and validation decisions are kept separate from the held-out locked test. Once a learned-policy configuration is selected on validation data, the checkpoint, feature schema, normalizer, policy settings and selection manifest are frozen before locked testing. Any later result must identify whether it comes from an analytic cache calculation, a simulated cache-policy experiment or a direct serving measurement. This prevents analytic payload estimates and masked-attention simulations from being misreported as measured GPU-memory savings or end-to-end latency.

3.6 Reproducibility, Artifact Management and Integrity Controls:

The foundations of reproducibility are built on deterministic seeds, immutable revisions for both models and datasets, the specification of pinned Python dependencies, policy descriptions within the code and comprehensive run manifests. A smallest run directory contains the runtime manifest, benchmark manifest & its result for formula-gate and core-test, attention profile metadata, raw result csv and figure plots and any other records. The drive is structured at top level as foundations, cache-core, T4 baseline, learned-policy, H100 locked-test & final-reporting inputs etc such that each plot or table reported in paper can be mapped back to its original run execution.

To avoid stale-code errors, the shared kvcore bundle version must be verified in the Colab bootstrap or re-extracted after updating the bundle, than just relying on the existence of a local kvcore directory. Explicit execution gates are added in the notebooks that record the hardware limitations as data. When put together, a scenario in which dataset configuration is not available, where GPU is not supported or Out-of-memory leads to results of the test than either missing or selectively discarded chunks of evidence.

Table 4. Current Colab-based experimental configuration and reporting controls.

Stage	Notebook or run area	Primary purpose	Required control or output
Foundations	00_foundations_environment_eda.ipynb	Validate GQA-aware KV accounting and audit the controlled 200-record corpus.	Formula-gate JSON, corpus manifest, EDA CSVs/figures and runtime manifest.
Cache core	01_evicting_cache_core_tests.ipynb	Verify eviction budgets, monotonic positions, full-cache equivalence and streaming decode.	Cache-core pass record, policy-budget CSV and decode trace.
T4 baselines	02_t4_ruler_baseline_sweeps.ipynb	Run gated controlled-corpus heuristic baselines and short attention profiling on T4.	Run gate, controlled baseline manifest, status/observed CSV and figures.
Learned-policy development	03_rtx_pro6000_longbench_learned_eviction.ipynb	Generate development labels, train/select the scorer on validation only, and evaluate validation policies on RTX PRO 6000.	Frozen checkpoint, feature schema, selection JSON, validation CSV and memory audit.
Locked evaluation	04_a100_locked_test_serving.ipynb	Evaluate the frozen scorer once on the 40 held-out controlled test records and collect A100 cache/latency instrumentation evidence.	Locked-test manifest, test-only results, complete resume journal, and artifact inventory; separate serving measurements remain disabled.
Reporting	05_results_pareto_and_reporting.ipynb	Aggregate observed artifacts and produce scoped A100 reporting tables and diagnostic figures.	Source CSVs, provenance summary, scoped A100 figures, and guarded Pareto output when full-cache retrieval quality is adequate.

4 Results

This unfinished draft Results lists the evidence from the completed run of Notebook 03 learned-policy development, separating out findings that were validation-only or expected through the coming locked-test and reporting stages. In the project artifact archive were stored the full run's Google Colab root, the checkpoints manifest, the model selection table and the corresponding validation-only column.

4.1 Learned-Scorer Training and Validation Selection

The trained scorer was trained using 48 oracle-label blocks derived from 12 controlled training documents and selected using 32 label blocks generated from a dedicated set of 8 validation documents. Crucially, the process of label generation and scorer validation did not involve the 40 documents reserved for testing. The chosen trial used a learning rate of 0.001, dropout of 0.10, weight decay of 0.0001 and 16-token blocks. Trial 0's validation NDCG@3 was 0.9780 compared with 0.9696 for the alternative trials.

The curve shows learning for training in the controlled dev set. Pairwise logistic loss for training reduced: from 0.689 at epoch-1 to 0.660 at epoch-10. Validation NDCG@3 increased 0.908 up to 0.978 at epoch 5 and stayed there. Validation pairwise agreement became: from 0.522 up to 0.870. The results show the models could learn the oracle block-ranking task. The results do not show the answers are of good quality or can be generalized out of the controlled corpus.

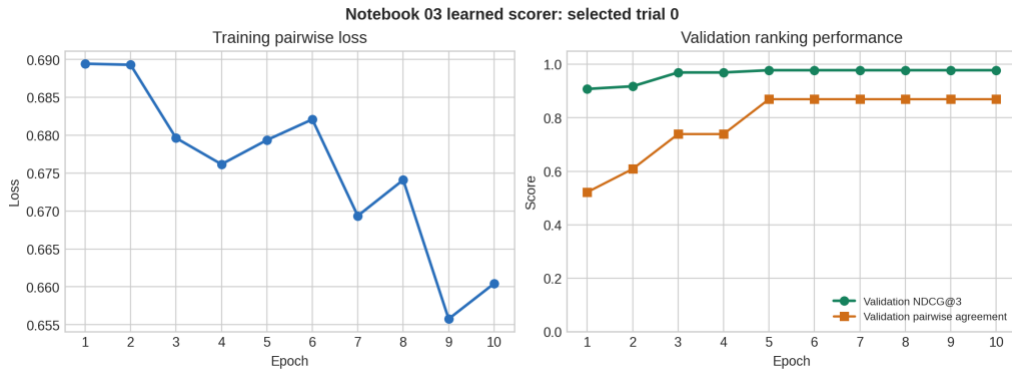


Figure 4. Training loss and validation-only ranking performance for the selected learned KV-cache scorer (Notebook 03).

4.2 Validation-Only Policy Evaluation

The 40 validation records, from the four planned context-length strata, were used to evaluate the checkpoint 40

validation records from four selected context-length strata to evaluate the checkpoint. All of the 680 recorded policy-by-budget evaluation rows had status `observed`. The learned-block policy reduced the physical KV-cache memory before eviction compared to the average of FIFO, sink-recent and H2O by 25.8% compared to their 24.2%. On average a small relative reduction in physical memory compared to FIFO, sink-recent and H2O was produced by the checkpoint while this checkpoint did not use further downstream processing like fine-tuning using RL. Result from quality is a restriction not claim of success. In this run, exact match is 0.0% as for all policy. Substring match was low for all policies with 10.0% for full cache and H2O, 7.5% for FIFO and sink-recent and 0.0% for learned-block. The above evidence from evaluation just proves the ability of scorer's block ranking capability and a minor physical memory saving, without showing signs of improving retrieval answer quality. This distinction has to be made in final copy of the report.

Policy	Observed rows	Mean KV-memory reduction	Mean substring match	Mean elapsed time (s)
Full cache	40	0.0%	10.0%	0.624
FIFO	160	24.2%	7.5%	0.643
Sink-recent	160	24.2%	7.5%	0.643
H2O	160	24.2%	10.0%	5.859
Learned block	160	25.8%	0.0%	0.731

Table 5. Validation-only policy evidence from the 40 controlled development records. These figures are not locked-test results.

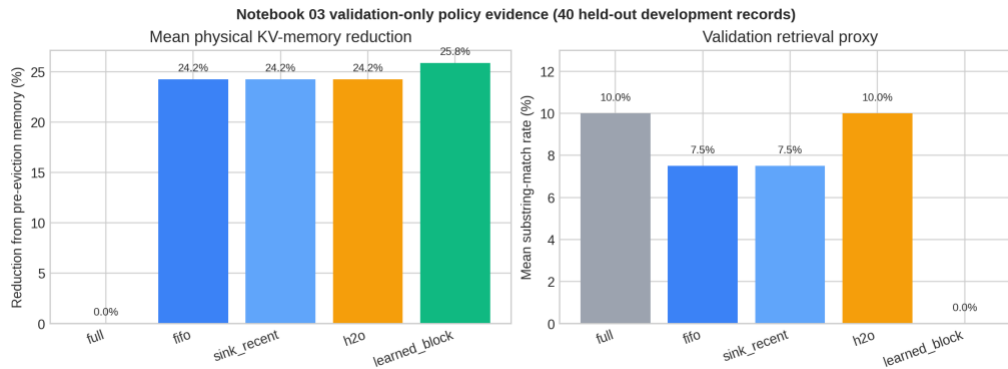


Figure 5. Validation-only physical KV-memory reduction and retrieval-proxy outcomes by policy (Notebook 03).

4.3 A100 Locked-Test Completion and Remaining Reporting Placeholder

Checkpoint of frozen Trial 0 with selected only validation (NDCG@3 = 0.9780) is tested once in Notebook 04 on its preassigned held-out partition of 40 records on an NVIDIA A100 of 80 GB of memory runtime. There existed total of 1000 measurements (policy-by-budget) conducted that had been pre-declared with complete Drive-based resume manifest along with a matching durable progress journal and matching final CSV exports, for which this locked-test was resumable. It is locked-test and NVIDIA A100 observation, not H100 evidence.

In the prompt, decode and scoring protocol employed, the run provided evidence towards cache or latency instrumentation. It did not support retrieval-preservation or superiority claim of learned policy. This score in Exact match was 0.0% and substring match 2.1%. Reference full cache also returned nearly zero retrieval quality. At 29.45% versus full cache we have the mean of observed physical KV-cache payload reduction within forced eviction subset. Aforementioned payload reduction is not measured end-to-end serving-memory/throughput result. Notebook 05 directly accumulated the Google Drive contents consisting of policy artefacts encountered above. 1,680 policy-by-budget rows resulted as compatible: the 680 validation-only RTX PRO 6000 rows found under Notebook 03 plus the 1,000 respective-locked A100 test rows obtained upon deduplication of the two Notebook 04 equivalent result exports. Notebook 05 was set at reporting scope with only 1,000 respective, locked-set A100 test rows included for purpose of figures and diagnostic aggregation such that respective quality-versus-budget and KV payload figures did not get muddled up between mixture of validation evidence from RTX systems and locked test on A100 systems. The raw routine returned two individual configurations at 1,024-token context of either as being non-dominated. But given the reference, it is merely given here as a diagnosis. A case having full reference cache running with the same protocol could not satisfy the retrieval quality required. So the meaning of policy ranking or effectiveness frontier cannot be conveyed through this result.

5 Conclusion

This report investigated whether trained, attention-aware eviction could improve the quality-memory trade-off for long-context inference. In the completed development process a deterministic controlled-retrieval corpus was created, cache accounting & eviction correctness checked and a light-weight block scorer trained and evaluated using only validation-based evidence to select a checkpoint.

The most striking current result is mixed. The learned scorer scored on ranking oracle cache blocks on the validation partition ($\text{NDCG}@3 = 0.9780$) and achieved the highest reduction in mean physical KV-memory among the considered policies (25.8%). At the same time it managed to generate no substring matches on the current validation retrieval proxy while full cache and H2O showed 10.0% matches each! This surprising gap shows that on its own ranking oracle cache block utility is not sufficient for aggressive eviction while still preserve downstream generated answer.

Evaluation of locked Notebook 04, after it was run to evaluate with the frozen selected checkpoint (contains 40 held-out records and 1,000 policy-by-budget observed measurements on A100). Notebook 05 correctly limited the aggregated rows from original Google Drive artifacts to compatible 1,680 and limited the figures and diagnostic for A100 to the 1,000 held-out A100 rows after filtering final copies. There were two non-dominated individual setting by direct Pareto method for generating non-dominated front but this is diagnostics and no finding on policy effective because full-cache retrieval quality was poor in the current prompt/decode/scoring protocol the A100 measurements will be reported as evidence of caching and latency instrumentation but not of learned policy effect (the main improvements for continuing this project after final submission involves revision of answer quality metric and protocol, extension of oracle labels to longer-range dependencies, use layer-level of chunk-based features and extend a versioned LongBench). For productization it would be necessary to carry out a serving system incorporating batching of requests, request scheduling, alongside operational visibility.

References

- Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., ... Li, J. (2024). LongBench: A bilingual, multitask benchmark for long context understanding. *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. <https://aclanthology.org/2024.acl-long.172/>
- Cai, Z., Zhang, Y., Gao, B., Liu, Y., Li, Y., Liu, T., Lu, K., Xiong, W., Dong, Y., Hu, J., & Xiao, W. (2024). PyramidKV: Dynamic KV cache compression based on pyramidal information funneling [Preprint]. arXiv. <https://arxiv.org/abs/2406.02069>
- Dasigi, P., Lo, K., Beltagy, I., Cohan, A., Smith, N. A., & Gardner, M. (2021). A dataset of information-seeking questions and answers anchored in research papers. *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. <https://aclanthology.org/2021.naacl-main.168/>
- Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., ... Vasic, P. (2024). The Llama 3 herd of models [Preprint]. arXiv. <https://arxiv.org/abs/2407.21783>
- Hsieh, C.-P., Sun, S., Krizan, S., Acharya, S., Rekesh, D., Jia, F., ... Ginsburg, B. (2024). RULER: What is the real context size of your long-context language models? [Preprint]. arXiv. <https://arxiv.org/abs/2404.06654>
- Huang, L., Cao, S., Parulian, N., Ji, H., & Wang, L. (2021). Efficient attentions for long document summarization. *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. <https://aclanthology.org/2021.naacl-main.409/>
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., & Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*. <https://arxiv.org/abs/2309.06180>
- Lhoest, Q., Villanova del Moral, A., Jernite, Y., Thakur, A., von Platen, P., Patil, S., ... Wolf, T. (2021). Datasets: A community library for natural language processing. *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. <https://aclanthology.org/2021.emnlp-demo.21/>
- Li, Y., Huang, Y., Yang, B., Venkitesh, B., Locatelli, A., Ye, H., Cai, T., Lewis, P., & Chen, D. (2024). SnapKV: LLM knows what you are looking for before generation [Preprint]. arXiv. <https://arxiv.org/abs/2404.14469>
- Liu, Z., Desai, A., Liao, F., Wang, W., Xie, V., Xu, Z., Kyrrilidis, A., & Shrivastava, A. (2023). Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. *Advances in Neural Information Processing Systems*, 36. https://proceedings.neurips.cc/paper_files/paper/2023/hash/a452a7c6c463e4ae8fbdc614c6e983e6-Abstract-Conference.html
- Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., & Hu, X. (2024). KIVI: A tuning-free asymmetric 2-bit quantization for KV cache. *Proceedings of the 41st International Conference on Machine Learning*. <https://arxiv.org/abs/2402.02750>
- Liu, X., Tang, Z., Dong, P., Li, Z., Liuyue, Li, B., Hu, X., & Chu, X. (2025). ChunkKV: Semantic-preserving KV cache compression for efficient long-context LLM inference. *Advances in Neural Information Processing Systems*, 38. https://proceedings.neurips.cc/paper_files/paper/2025/hash/2987f911151b39cd3a1761e212319e8e-Abstract-Conference.html
- Moschella, L., Manduchi, L., & Sener, O. (2026). Learning to evict from key-value cache [Preprint]. arXiv. <https://arxiv.org/abs/2602.10238>
- Pope, R., Douglas, S., Chowdhery, A., Devlin, J., Bradbury, J., Levskaya, A., ... Dean, J. (2023). Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems*, 5. https://proceedings.mlsys.org/paper_files/paper/2023/hash/83a2b5b4f4e1dddf4a49a66d1f4d9f2e-Abstract-mlsys2023.html
- Qwen Team. (2024). Qwen2.5 technical report [Preprint]. arXiv. <https://arxiv.org/abs/2412.15115>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30. https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html
- Xiao, G., Tian, Y., Chen, B., Han, S., & Lewis, M. (2023). Efficient streaming language models with attention sinks [Preprint]. arXiv. <https://arxiv.org/abs/2309.17453>
- Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., ... Chen, B. (2023). H2O: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 36. https://proceedings.neurips.cc/paper_files/paper/2023/hash/b5a2b0f8b5f5ec6a2c8f4e9b8c5cc5af-Abstract-Conference.html